

AEDS 数据库高级机制实验验证汇总报告

实验日期：2026 年 6 月 19 日 实验小组：第 23 组

1 索引设计与性能实验

1.1 核心查询语句 (Q2: 工况条件筛选)

```
1 -- 在 alpha=0, Re=100000 条件下筛选 Cd <= 0.02 的翼型, 并按升阻比降序排列
2 SELECT a.airfoil_code, av.version_no, pr.cl, pr.cd, pr.l_over_d
3 FROM airfoil a
4 JOIN airfoil_version av ON av.airfoil_id = a.airfoil_id
5 JOIN performance_record pr ON pr.version_id = av.version_id
6 JOIN experiment_condition ec ON ec.condition_id = pr.condition_id
7 WHERE ec.alpha_deg = 0 AND ec.reynolds_number = 100000
8 AND pr.cd <= 0.02 AND av.is_current = true
9 ORDER BY pr.l_over_d DESC LIMIT 50;
```

Listing 1: 工况条件筛选核心 SQL

1.2 执行计划文本输出与性能对比

实验阶段对比结果

- 无索引情况 (Phase 0): 执行耗时 **3.079 ms**。全表扫描 performance_record (9654 行) 及 airfoil_version。
- 单列索引 (Phase 1): 执行耗时 **0.618 ms**。建立了 performance_record(condition_id) 索引, 避免了全表扫描, 性能提升 **5.0x**。
- 复合索引 (Phase 2): 执行耗时 **0.579 ms**。建立了 performance_record(condition_id, l_over_d DESC) 复合索引, 消除了排序开销, 性能提升 **5.3x**。

```
1 Limit (actual time=0.579..0.582 rows=50 loops=1)
2   -> Nested Loop (actual time=0.578..0.580 rows=50 loops=1)
3     -> Index Scan using idx_perf_cond_lod on performance_record (
4         actual time=0.015..0.245 rows=400 loops=1)
5         Index Cond: (condition_id = '...')
6     -> Index Scan using idx_version_is_current on airfoil_version (
7         actual time=0.001..0.001 rows=1 loops=400)
8 Planning Time: 1.909 ms | Execution Time: 0.579 ms
```

Listing 2: Phase 2 最佳执行计划文本 (EXPLAIN ANALYZE)

1.3 索引选择理由与思辨

为什么选择复合索引 (`condition_id, l_over_d`)?

- 谓词对齐：查询频繁按工况 (`condition_id`) 过滤，单列索引已能显著减少扫描量。
- 消除排序：增加 `l_over_d` 作为复合列并设置 DESC 排序，使数据库能直接从索引有序读取前 50 条，彻底消除了内存或磁盘中的外部排序 (External Sort) 步骤。
- 为什么不选其他字段?: `cl` 和 `cd` 虽在 SELECT 中出现，但非 WHERE 过滤的主键，建索引收益远低于 `condition_id`。

2 事务与并发控制实验

2.1 场景：批量导入回滚 (原子性)

实验过程：调用 `governance.import_performance_batch` 存储过程，传入包含非法 `Cd < 0` 记录的 JSON。

原子性验证结果

NOTICE: Expected rollback triggered: cd must be ≥ 0

验证结果: Before Count = 9654, After Count = 9654。

结论： 第一条合法记录随第二条非法记录一同回滚，实现了事务原子性。

2.2 场景：乐观锁并发修改 (一致性)

实验过程：模拟两个用户基于同一 `xmin` 版本号进行修改。

- 用户 A：提交修改，`xmin` 匹配成功，数据更新。
- 用户 B：随后提交，但其持有的 `xmin` 已过期，数据库返回 `success=false` 并提示版本冲突。

3 高级机制：视图、触发器与存储过程

3.1 视图：统一展示 “当前有效版本”

```
1 CREATE VIEW v_current_airfoil_version AS
2 SELECT airfoil_code, name, version_no FROM airfoil_version
3 WHERE is_current = true AND status = 'valid' AND is_deleted = false;
```

实际意义：屏蔽了历史修订版本和软删除数据，为所有工程报表提供唯一可信的数据源。

3.2 触发器：自动写入修改日志

```
1 CREATE TRIGGER trg_log_change_performance AFTER INSERT OR UPDATE
2 ON performance_record FOR EACH ROW EXECUTE FUNCTION governance.log_change()
;
```

实际意义：实现了非侵入式的审计追踪，任何对性能数据的微调都会被记录在 `change_log` 中。

3.3 存储过程：批量导入与合法性检查

实现：`governance.import_performance_batch` 封装了 JSON 解析、跨表关联以及 $Cd \geq 0$ 的物理校验。
实际意义：将复杂的业务逻辑从应用层下沉至数据库层，保证了数据入库的绝对合法性。

4 实验检查汇总

检查项	要求	状态
索引实验	包含无索引、单列、复合对比，含执行计划与理由	100%
事务实验	包含批量导入非法回滚、并发冲突验证	100%
高级机制	同时实现了视图、触发器和存储过程	100%